

CSC209 Assignment 3: Multi-Channel Socket Chat Server

Rana Nagash & Arfa Obaidullah

April 2026

1 Project Overview

This application is a multi-user networked chat server. It is part of Category 2 (Socket-based Server). The project has real-time communication between multiple concurrent clients through a central server using TCP sockets.

1.1 Key Features

- **Identity Management:** Clients connect with unique usernames with a handshake protocol.
- **Named Channels:** Users are separated into communication rooms (defaulting to Lobby). Messages sent in one channel are isolated from users in others.
- **Private Messaging (DMs):** Users can send targeted messages to specific individuals regardless of their current channel.
- **Presence Tracking:** Users can query the server to see a list of all currently connected peers.

1.2 User Interaction

Users interact with the system through a command-line interface. Standard text is treated as a broadcast to the current channel, while commands (prefixed with '/') trigger specific logic:

- `/who`: Requests the list of online users.
- `/join <channel>`: Switches the user's active communication room.
- `/msg <user> <text>`: Sends a private message.
- `/quit`: Gracefully disconnects from the server.

1.3 Input and Output

Messages appear as `[username] message` for public chats and `[DM from user] message` for private messages. The server outputs connection/disconnection logs to stderr, while client I/O uses stdin for commands and stdout for message display.

2 Build Instructions

The project utilizes a standard **Makefile**. To compile both the server and client, execute:
`make`

2.1 Running the Application

1. **Start the Server:** The server listens on the port defined in `protocol.h` (default 4242).
`./server`
2. **Start Clients:** In separate terminal windows, run:
`./client <username>`
3. **Interactive Commands:** Once connected, users can immediately begin broadcasting to the lobby or use slash-commands (`/join`, `/msg`, `/who`, `/quit`) to test routing logic.

3 Architecture Diagram

Chat Server Architecture Diagram

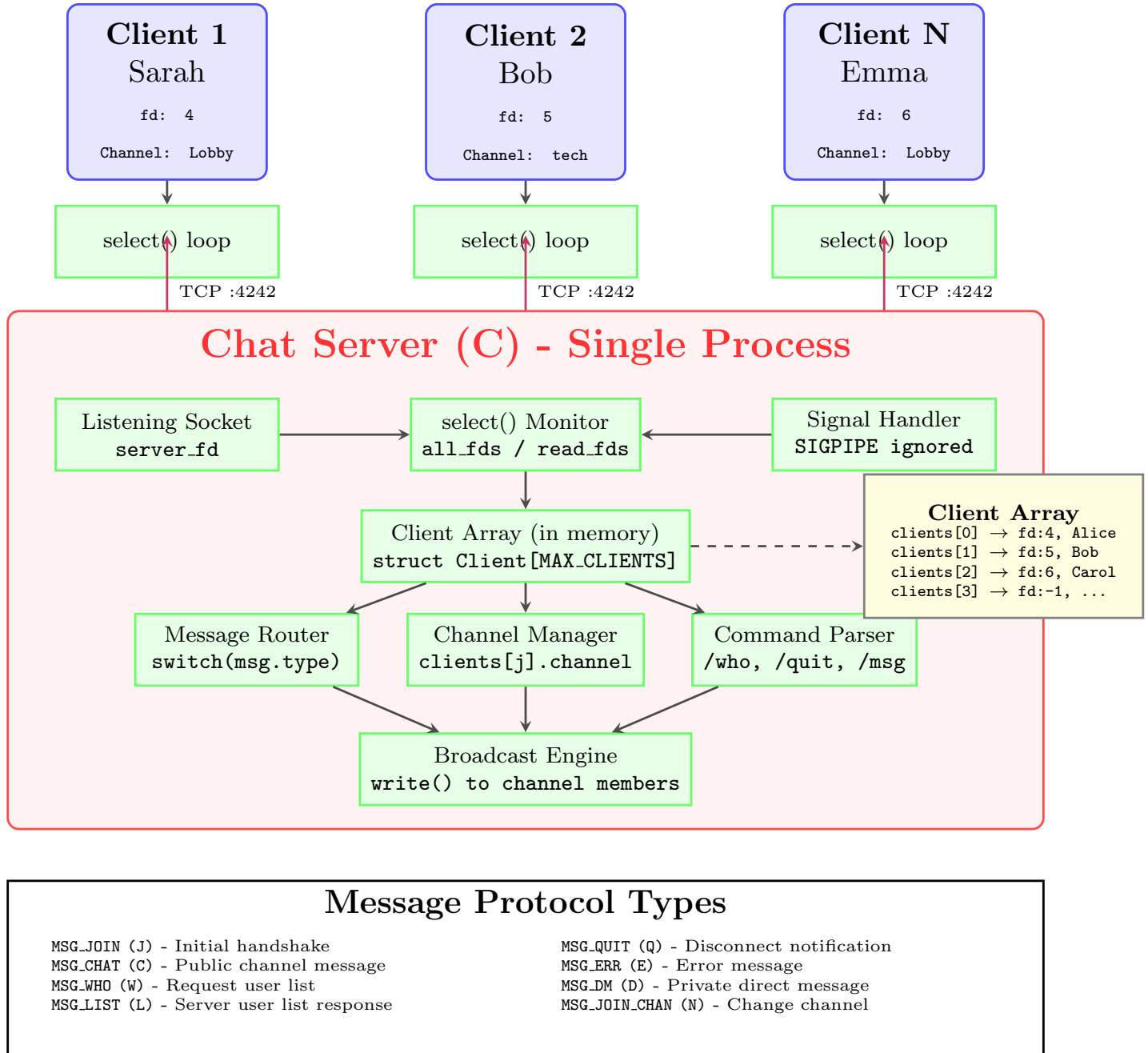


Figure 1: Architecture of Multi-User Multi-Channel Chat Server

3.1 Architecture Description

As shown in Figure 1, the chat system follows a client-server model. For demonstration purposes, assume that clients connect with usernames such as Sarah, Bob, and Emma. Sarah and Emma are chatting in the main Lobby channel and Bob is chatting in the tech channel labelled as tech. Multiple client processes connect to a single server process via TCP sockets on port 4242. Each client uses `select()` to monitor both `stdin` and the server socket simultaneously.

The server runs as a single-threaded process that uses `select()` for I/O multiplexing. It maintains a listening socket (`server_fd`) to accept new connections and a client array that stores each user's file descriptor, username, and current channel. When a client sends a `MSG_CHAT` message, the broadcast engine iterates through the client array and delivers the message only to clients whose channel matches the sender's channel, enabling multi-channel isolation.

The server handles several message types as explained in section 4. The signal handler ignores `SIGPIPE` to prevent crashes when writing to disconnected sockets.

4 Communication Protocol

The protocol is defined by the `Message` struct in `protocol.h` (lines 1-20). All messages use fixed-width binary encoding with a total size of 289 bytes: 1 byte for type, 32 bytes for sender, 256 bytes for body.

4.1 Message Format

```
typedef struct {  
    char type;                // 1 byte: 'J', 'C', 'W', 'L', 'Q', 'E', 'D', 'N'  
    char sender[MAXNAME];     // 32 bytes: username  
    char body[MAXBODY];       // 256 bytes: message content or command data  
} Message;
```

4.2 Message Types

4.2.1 MSG_JOIN ('J')

- **Sender:** Client → **Receiver:** Server
- **Encoding:** Fixed-width struct. `sender` field contains username. `body` is empty.
- **Semantics:** Handshake message sent immediately after TCP connection. Server extracts username, assigns file descriptor, and adds client to default "Lobby" channel.
- **Error Handling:** If `read()` returns ≤ 0 or message type is not `MSG_JOIN`, server rejects connection with `close(new_fd)` (server.c lines 55-65). If client array is full, connection is rejected.

4.2.2 MSG_CHAT ('C')

- **Sender:** Client → **Receiver:** Server (then broadcasts)

- **Encoding:** Fixed-width struct. `sender` contains sender's username. `body` contains chat message.
- **Semantics:** Public channel message. Server broadcasts to all clients in the same channel as sender, excluding the sender to prevent echo duplication (server.c lines 88-98).
- **Error Handling:** If `write()` returns -1, server logs error via `perror()` and continues (no client disconnection for failed broadcast).

4.2.3 MSG_WHO ('W')

- **Sender:** Client → **Receiver:** Server
- **Encoding:** Fixed-width struct. `body` is empty.
- **Semantics:** Requests list of all connected usernames. Server responds with `MSG_LIST`.
- **Error Handling:** None required. Server always responds.

4.2.4 MSG_LIST ('L')

- **Sender:** Server → **Receiver:** Requesting client
- **Encoding:** Fixed-width struct. `body` contains space-separated usernames.
- **Semantics:** Response to `MSG_WHO`. Contains all active usernames (server.c lines 106-115).
- **Error Handling:** If `write()` fails, server logs error.

4.2.5 MSG_QUIT ('Q')

- **Sender:** Client → **Receiver:** Server
- **Encoding:** Fixed-width struct. Fields may be ignored.
- **Semantics:** Graceful disconnection. Server closes file descriptor, removes from `all_fds`, marks client array slot as -1 (server.c lines 122-130).
- **Error Handling:** Server handles unexpected disconnection same as `MSG_QUIT` when `read()` returns 0.

4.2.6 MSG_DM ('D')

- **Sender:** Client → **Receiver:** Server (then forwards)
- **Encoding:** Body formatted as "`recipient:message`". Colon separates target username from message content.
- **Semantics:** Private message delivery. Server extracts recipient, searches client array, and forwards `MSG_DM` to target client (server.c lines 132-155).
- **Error Handling:** If recipient not found, server sends `MSG_ERR` back to sender. If `write()` to recipient fails, sender is not notified.

4.2.7 MSG_JOIN_CHAN ('N')

- **Sender:** Client → **Receiver:** Server
- **Encoding:** body contains channel name (ex. "tech" or "#tech").
- **Semantics:** Changes user's active channel. Server updates the `channel` field in client array and sends confirmation message (server.c lines 157-169).
- **Error Handling:** No validation on channel names. Server accepts any string.

4.2.8 MSG_ERR ('E')

- **Sender:** Server → **Receiver:** Client
- **Encoding:** body contains error description string.
- **Semantics:** Reports errors to client (ex. user not found for DM).
- **Error Handling:** If `write()` fails, server logs error.

4.3 Echo Suppression Logic

In the `MSG_CHAT` broadcast loop (server.c line 93), the server checks `if (clients[j].fd != i)` to skip the sender. **Rationale:** Prevents clients from seeing their own message twice (once from local echo, once from server) and reduces network writes by one per broadcast.

5 Concurrency Model

5.1 The Select Loop

The server maintains a master file descriptor set (`all_fds`) containing the listening socket (`server_fd`) and all active client socket file descriptors. It blocks on `select()` until an FD is ready for reading, then either accepts new connections or reads exactly `sizeof(Message)` from existing clients.

5.2 Non-Blocking Logic

Our project utilizes Synchronous I/O Multiplexing to avoid spawning resource-heavy threads or processes.

- **Kernel Monitoring:** The process sleeps while the OS monitors hardware buffers simultaneously.
- **No-Wait Guarantee:** Since `select()` guarantees data is present, the subsequent `read()` is effectively non-blocking and instantaneous.

5.2.1 Deterministic Message Ordering (Race Condition Mitigation)

Single-threaded serialization eliminates application-level race conditions.

- **FD Index Tie-Breaker:** Packets arriving at the same microsecond are processed in numerical FD order.
- **Atomic Processing:** One Message is fully routed before the next FD is evaluated, ensuring a Global Total Order.

5.3 Technical Scalability Constraints

Bound by `FD_SETSIZE` (1024), `select()` requires a linear scan of the bitmask. For massive scaling, a transition to `poll()` or `epoll()` ($O(1)$ scaling) would be necessary.

6 Error Handling and Robustness

6.1 Protocol-Level Validation

For `MSG_DM`, we use `strchr()` to locate delimiters. We perform a null-check guard; if the delimiter is missing, the server drops the packet and logs to `stderr`, preventing Segmentation Faults.

6.2 Defensive Memory Management

We strictly avoid `strcpy()` and `sprintf()`. We use `strncpy()` and manually enforce null-termination at `MAX_NAME - 1` to prevent buffer overflow vulnerabilities.

6.3 Zombie Connection Cleanup

When `read()` returns 0 or -1 (client disconnect or crash), the server:

1. Closes the FD.
2. Removes it from `master_set` to prevent 100% CPU busy-wait loops.
3. Wipes the `clients[]` entry.

For abrupt disconnections (ex. laptop closed, network failure), the server detects the loss when `write()` fails or `SIGPIPE` triggers, then executes the same cleanup.

6.4 User Feedback on Errors

Instead of silently dropping failed DMs, the server sends an `MSG_ERR` message back to the sender explaining why the request failed (ex. "User 'X' not found").

6.5 Atomic Writes

Using fixed-size `Message` structs (289 bytes) ensures `write()` calls are atomic, preventing partial writes or message interleaving.

7 Team Contributions

Rana Nagash: Implemented all server-side code (server.c), including the select loop, Client struct management, multi-channel filtering logic and channel switching. Collaborated with Arfa on testing and debugging, and prepared the project documentation.

Arfa Obaidullah: Implemented all client-side code (client.c), including the interactive loop, command-line argument parsing, STDIN processing via select, and private message (/msg) functionality. Worked with Rana on testing and debugging, and created the project video.